INSTITUT NATIONAL DES POSTES ET TELECOMMUNICATIONS

Filière: ASEDS

Rapport de devoir 1:

Devoir 1: Conteneurisation d'une Application Web

Année academique: 2026-2027

Réalisé par :

- EL-MARRHOUB Anas

Table des matières

1 Introduction et Contexte du Projet **3**
1.1 Problématique industrielle : L'entreprise NeoTech . . . 3
1.2 Architecture logicielle de l'application Travelo . . . 3
1.3 Clonage du projet et initialisation de l'environnement . . . 3

2 Volet 1 : Conteneurisation de l'Application avec Docker **4**
2.1 Q1. Création du réseau Docker personnalisé . . . 4
2.2 Q2. Conteneurisation de la Base de Données MySQL . . . 5
2.2.1 a. Persistance des données et justification du stockage . . . 5
2.2.2 b. Instanciation et raccordement au réseau . . . 6
2.2.3 Preuve expérimentale de la persistance des données . . . 7
2.3 Q3 & Q4. Dockerfile du Projet Backend (Spring Boot) et Bonnes Pratiques . . . 8
2.3.1 Code complet du Dockerfile Backend . . . 8
2.3.2 Fichier .dockerignore Backend . . . 8
2.3.3 Explication détaillée des bonnes pratiques appliquées (Q4) . . . 10
2.4 Q5. Cycle de vie de l'image et du conteneur Backend . . . 11
2.4.1 a. Création de l'image en local . . . 11
2.4.2 b. Scan des vulnérabilités (DevSecOps) et actions de remédiation . . . 11
2.4.3 c. Publication de l'image sur Docker Hub . . . 12
2.4.4 d. Instanciation du conteneur Backend en local . . . 13
2.4.5 e. Inspection du conteneur Backend . . . 14
2.4.6 f. Consultation des journaux d'exécution (logs) et test HTTP . . . 14
2.5 Q6. Conteneurisation du Projet Frontend (React) . . . 16
2.5.1 6.1 (Q3) Code source du Dockerfile Frontend . . . 16
2.5.2 Fichier .dockerignore Frontend . . . 16
2.5.3 6.2 (Q4) Bonnes pratiques spécifiques appliquées au Frontend . . . 17
2.5.4 6.3 (Q5a) Build local de l'image Frontend . . . 17
2.5.5 6.4 (Q5b) Scan de vulnérabilités et remédiation DevSecOps . . . 17
2.5.6 6.5 (Q5c) Tag et publication sur Docker Hub . . . 18
2.5.7 6.6 (Q5d) Instanciation du conteneur Frontend . . . 19
2.5.8 6.7 (Q5e) Inspection du conteneur Frontend . . . 19
2.5.9 6.8 (Q5f) Journaux d'exécution du Frontend . . . 19
2.6 Q7. Validation du fonctionnement de l'application 3-tiers . . . 20
2.7 Q8. Nettoyage des conteneurs en exécution . . . 20
2.8 Q9. Orchestration locale avec Docker Compose . . . 20
2.8.1 a. Fichier docker-compose.yml . . . 20
2.8.2 b. Commande d'exécution . . . 21
2.8.3 c. Validation du fonctionnement . . . 22
2.9 Q10. Nettoyage complet de l'environnement hôte . . . 23
2.10 Q11. Mise en place d'un Registre Privé Docker avec Interface Graphique . . . 23
2.10.1 Alimentation du registre privé en images locales . . . 25
2.11 Q12. Redéploiement via Docker Compose depuis le Registre Privé . . . 26
2.11.1 Vérification du catalogue via l'API REST v2 . . . 27
2.11.2 Purge des images locales et redéploiement autonome . . . 27

3 Volet 2 : Orchestration des Conteneurs avec Docker Swarm **29**
3.1 Initialisation du cluster Swarm . . . 29
3.2 Q13, Q14, Q15 & Q16. Conception de la Stack Docker Swarm . . . 29
3.2.1 Fichier complet docker-compose.yml pour la Stack Swarm . . . 29
3.2.2 Justification des exigences pédagogiques (Q13 à Q16) . . . 31

3.2.3 Commandes de déploiement et d'inspection de la Stack 32
3.3 Q17. Configuration du Reverse Proxy (Nginx / Traefik) 34
3.3.1 Fichier de configuration Nginx (nginx.conf) 34
3.4 Q18. Validation finale du Cluster Swarm dans le navigateur 36

4 Conclusion Générale **37**

1 Introduction et Contexte du Projet

1.1 Problématique industrielle : L'entreprise NeoTech

L'entreprise **NeoTech** est une jeune structure technologique spécialisée dans le développement de services logiciels innovants. Bien que le code produit soit de haute qualité fonctionnelle, les équipes d'ingénierie se heurtent régulièrement à un frein opérationnel majeur : le manque de parité et les dérives de configuration entre les postes de développement, les serveurs d'intégration et l'environnement de production. Ce phénomène entraîne des incompatibilités imprévues, des retards de livraison et alourdit la charge de maintenance.

Pour adresser ce verrou technique, NeoTech a mandaté la réalisation d'un **Proof of Concept (PoC)** centré sur la conteneurisation intégrale et l'orchestration en grappe (*cluster*) de son application web phare : **Travelo**.

1.2 Architecture logicielle de l'application Travelo

L'application Travelo est une plateforme de réservation de vols aériens construite selon un modèle 3-tiers découplé :

- **Tier Présentation (Frontend) :** Interface monopage (*Single Page Application*) développée en React, offrant un moteur interactif de recherche de trajets.
- **Tier Logique Métier (Backend) :** API RESTful conçue avec le framework Java Spring Boot (v3.4), exposant les endpoints de consultation de vols et de réservation, et s'appuyant sur l'ORM Hibernate / Spring Data JPA.
- **Tier Persistance (Database) :** Base de données relationnelle MySQL 8.0 articulée autour du schéma `travelo_db`.

1.3 Clonage du projet et initialisation de l'environnement

L'expérimentation débute par la récupération du code source sur le dépôt GitHub de l'entreprise et la vérification préalable de la conformité du moteur Docker de la station de travail.

```
git clone https://github.com/dall49/travelo-app.git
cd travelo-app
docker --version && docker compose version
```

Explication des commandes

- `git clone ...` : Clone l'intégralité du code source, de l'historique et des branches du projet dans le sous-dossier local `travelo-app`.
- `docker -version` : Affiche la version courante du démon Docker Engine (**v29.4.1**, build 055a478).
- `docker compose version` : Valide la disponibilité du plugin Compose officiel (**v5.1.3**).

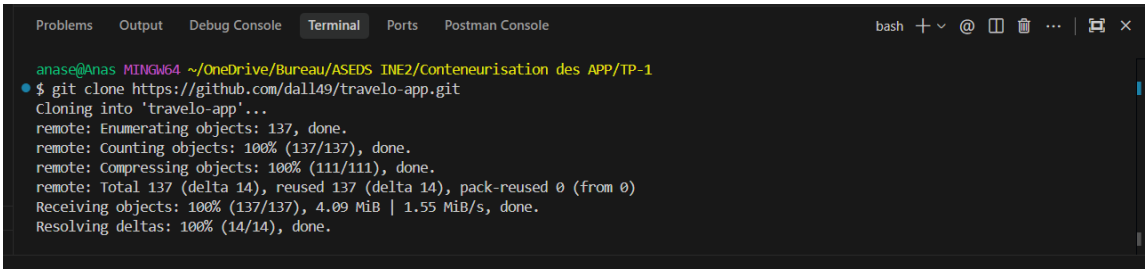


FIGURE 1 – Clonage du code source de l'application Travelo depuis le dépôt distant

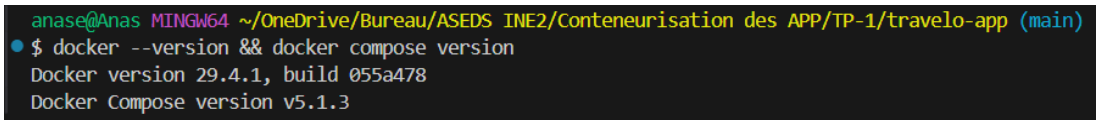


FIGURE 2 – Vérification des versions du moteur Docker et du composant Docker Compose

2 Volet 1 : Conteneurisation de l'Application avec Docker

2.1 Q1. Création du réseau Docker personnalisé

La communication entre les trois conteneurs nécessite un support réseau sécurisé et performant. Nous instancions un réseau pont (*bridge*) personnalisé :

```
docker network create --driver bridge travelo-net
docker network inspect travelo-net
```

Explication détaillée des attributs CLI

- `network create` : Sous-commande ordonnant au démon Docker la création d'un pont logiciel virtuel isolé.
- `-driver bridge` : Sélectionne le pilote pont (*bridge*). Il instancie une interface réseau commutée virtuelle sur l'hôte, isolant le trafic réseau des conteneurs rattachés.
- `travelo-net` : Nom arbitraire et signifiant assigné au réseau.
- `network inspect` : Extrait la configuration technique détaillée (IPAM, sous-réseau, passerelle, conteneurs connectés) au format JSON.

Justification Technique & Architecturale

Pourquoi un réseau bridge personnalisé plutôt que le réseau bridge par défaut ?

1. **Résolution DNS automatique par nom de conteneur :** Sur un pont personnalisé, Docker intègre un résolveur DNS interne. Le conteneur backend peut joindre la base de données directement via l'alias d'hôte `database` ou `travelo-database`, sans se soucier de son adresse IP dynamique. Le réseau bridge par défaut (`bridge`) n'offre aucune résolution DNS automatique (il imposerait l'usage de liaisons obsolètes `-link` ou d'adresses IP fixes).
2. **Meilleure isolation de sécurité :** Seuls les conteneurs expressément rattachés à `travelo-net` peuvent converser, empêchant toute écoute ou collision avec des conteneurs tiers du système.
3. **Attachement et détachement à chaud :** Il est possible de connecter ou déconnecter des conteneurs d'un réseau personnalisé à chaud sans nécessiter leur arrêt.

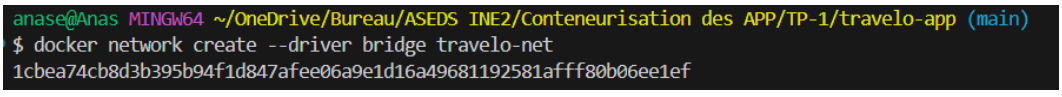


FIGURE 3 – Exécution de la commande de création du réseau personnalisé `travelo-net`

```
docker network inspect travelo-net

 {
     "Name": "travelo-net",
     "Id": "1cbea74cb8d3b395b94f1d847afee06a9e1d16a49681192581afff80b06ee1ef",
     "Created": "2026-10-24T11:20:58.435417921Z",
     "Scope": "local",
     "Driver": "bridge",
     "EnableIPv4": true,
     "EnableIPv6": false,
     "IPAM": {
         "Driver": "default",
         "Options": {},
         "Config": [
             {
                 "Subnet": "172.22.0.0/16",
                 "Gateway": "172.22.0.1"
```

FIGURE 4 – Inspection JSON de `travelo-net` : sous-réseau alloué `172.22.0.0/16` et passerelle `172.22.0.1`

2.2 Q2. Conteneurisation de la Base de Données MySQL

2.2.1 a. Persistance des données et justification du stockage

Par conception, le système de fichiers d'un conteneur Docker est éphémère : toute donnée écrite dans la couche d'écriture (*writable layer*) est définitivement détruite lors de la suppression du conteneur. Une base de données d'entreprise requiert une persistance absolue et indépendante du cycle de vie du conteneur.

Solution	Mécanisme technique	Avantages majeurs	Inconvénients
Docker Named Volume	Géré exclusivement par Docker dans `/var/lib/docker/volumes/`	Totalement portable, indépendant de l'hôte, haute performance I/O, sécurisé	Non directement explorable sans droits d'administration
Bind Mount	Montage direct d'un dossier hôte arbitraire dans le conteneur	Pratique en phase de dev pour éditer du code source	Dépend de l'OS hôte, risques de conflits de droits (UID/GID), latence WSL2
tmpfs Mount	Montage éphémère en mémoire vive (RAM) uniquement	Vitesse I/O maximale, sécurité (aucune trace disque)	Aucune persistance (perte totale des données à l'extinction)

TABLE 1 – Comparaison des mécanismes de persistance Docker pour un SGBD

Justification Technique & Architecturale

Pour le déploiement du SGBD MySQL, le **Volume nommé Docker** (`mysql-data`) constitue le choix le plus adéquat. Il garantit une gestion transparente par Docker Engine, des performances d'écriture optimales pour les moteurs transactionnels comme InnoDB, et immunise les données contre les suppressions accidentelles de l'hôte ou les différences de systèmes de fichiers entre Windows et Linux.

```
1 docker volume create mysql-data
```

- `volume create` : Crée une zone de stockage gérée au sein du sous-système de virtualisation Docker.
- `mysql-data` : Identifiant unique du volume dédié à la persistance d'InnoDB.

```
$ docker volume create mysql-data
mysql-data
```

FIGURE 5 – Création du volume persistant dédié `mysql-data`

2.2.2 b. Instanciation et raccordement au réseau

```
docker run -d --name travelo-database \
  --network travelo-net \
  -e MYSQL_ROOT_PASSWORD=chaima2003 \
  -e MYSQL_DATABASE=travelo_db \
  -v mysql-data:/var/lib/mysql \
  --restart unless-stopped \
  mysql:8.0
```

Explication détaillée des attributs CLI

- `-d` (*detached*) : Libère le terminal en exécutant le processus en arrière-plan.
- `-name travelo-database` : Attribue un nom réseau univoque au conteneur pour la résolution DNS.
- `-network travelo-net` : Attache le conteneur au réseau privé créé en Q1.
- `-e MYSQL_ROOT_PASSWORD=chaima2003` : Définit le mot de passe d'administration MySQL.
- `-e MYSQL_DATABASE=travelo_db` : Ordonne à MySQL d'initialiser automatiquement le schéma de l'application dès le premier lancement.
- `-v mysql-data:/var/lib/mysql` : Associe le volume nommé au répertoire interne où MySQL stocke ses tables et journaux de transactions.
- `-restart unless-stopped` : Politique de redémarrage automatique en cas de crash intempestif.
- `mysql:8.0` : Image officielle du SGBD téléchargée depuis Docker Hub.

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main)
$ docker run -d --name travelo-database \
  --network travelo-net \
  -e MYSQL_ROOT_PASSWORD=chaima2003 \
  -e MYSQL_DATABASE=travelo_db \
  -v mysql-data:/var/lib/mysql \
  --restart unless-stopped \
  mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
```

FIGURE 6 – Téléchargement de l'image officielle `mysql:8.0` et instanciation du conteneur

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main)
$ docker ps
CONTAINER ID   IMAGE                    COMMAND                  CREATED             STATUS              PORTS
                NAMES
1490cb9e1ab4   mysql:8.0                "docker-entrypoint.s…"   About a minute ago   Up About a minute   3306/tcp, 33060/tcp
                travelo-database
ba8555952f3b   node-mongo-mvc-nodeapp   "docker-entrypoint.s…"   4 months ago        Up 13 minutes       0.0.0.0:3000->3000/tcp, [::]:30
00->3000/tcp    node_mongodb
7f7ee3808c6d   mongo:8.0                "docker-entrypoint.s…"   4 months ago        Up 13 minutes       0.0.0.0:27017->27017/tcp, [::]:
27017->27017/tcp   mongodb
```

FIGURE 7 – Vérification du statut opérationnel (`Up About a minute`) via `docker ps`

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main)
$ docker exec -it travelo-database mysql -uroot -pchaima2003 -e "SHOW DATABASES;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| travelo_db         |
+--------------------+
```

FIGURE 8 – Validation de l'initialisation du schéma `travelo_db` par exécution SQL en conteneur

2.2.3 Preuve expérimentale de la persistance des données

Afin d'apporter la preuve irréfutable de la persistance, nous insérons un enregistrement dans une table personnalisée `test`, puis nous détruisons brutalement le conteneur à l'aide du flag `-f` (*SIGKILL*), avant d'en recréer un nouveau rattaché au même volume.

```
docker exec -it travelo-database mysql -uroot -pchaima2003 \
  -e "USE travelo_db; CREATE TABLE test(id INT); INSERT INTO test VALUES (1);"
docker rm -f travelo-database
docker run -d --name travelo-database --network travelo-net \
  -e MYSQL_ROOT_PASSWORD=chaima2003 -e MYSQL_DATABASE=travelo_db \
  -v mysql-data:/var/lib/mysql mysql:8.0
docker exec -it travelo-database mysql -uroot -pchaima2003 \
  -e "USE travelo_db; SELECT * FROM test;"
```

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main)
$ docker exec -it travelo-database mysql -uroot -pchaima2003 \
  -e "USE travelo_db; CREATE TABLE test(id INT); INSERT INTO test VALUES (1);"
mysql: [Warning] Using a password on the command line interface can be insecure.

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug travelo-database
    Learn more at https://docs.docker.com/go/debug-cli/
```

FIGURE 9 – Insertion d'une donnée de contrôle dans la table `test`

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main)
$ docker rm -f travelo-database
travelo-database
```

FIGURE 10 – Suppression forcée du conteneur MySQL (`docker rm -f`)

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main)
$ docker run -d --name travelo-database \
  --network travelo-net \
  -e MYSQL_ROOT_PASSWORD=chaima2003 \
  -e MYSQL_DATABASE=travelo_db \ …
  mysql:8.0
df50df4333e0e291fce9968d13c8bb22a48df4779cd43cd40fcd4df94a41414b

anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main)
$ docker exec -it travelo-database mysql -uroot -pchaima2003 \
  -e "USE travelo_db; SELECT * FROM test;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+------+
| id   |
+------+
|    1 |
+------+
```

FIGURE 11 – Recréation du conteneur et extraction de la donnée : la persistance est démontrée

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main
$ docker network inspect travelo-net
        "Name": "travelo-net",
        "Id": "1cbea74cb8d3b395b94f1d847afee06a9e1d16a49681192581afff80b06ee1ef",
        "Created": "2026-10-24T11:20:58.435417921Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.22.0.0/16",
                    "Gateway": "172.22.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
```

FIGURE 12 – Confirmation de l'intégration du conteneur `travelo-database` dans le réseau `travelo-net`

2.3 Q3 & Q4. Dockerfile du Projet Backend (Spring Boot) et Bonnes Pratiques

2.3.1 Code complet du Dockerfile Backend

Le fichier `Dockerfile` conçu pour le service Java Backend applique rigoureusement les standards de l'art :

```
FROM eclipse-temurin:23-jdk-alpine AS builder
WORKDIR /app

COPY .mvn/ .mvn/
COPY mvnw pom.xml ./

RUN sed -i 's/\r$//' mvnw && chmod +x mvnw
RUN ./mvnw dependency:go-offline -B

COPY src/ ./src/
RUN ./mvnw clean package -DskipTests -B

FROM eclipse-temurin:23-jre-alpine
WORKDIR /app

RUN apk update && apk upgrade --no-cache
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

COPY --from=builder /app/target/*.jar app.jar

USER appuser

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

2.3.2 Fichier .dockerignore Backend

```
target/
build/
```

```
.idea/
.vscode/
*.iml
.DS_Store
Thumbs.db
.git/
.gitignore
.gitattributes
*.log
*.tmp
README.md
*.md
```

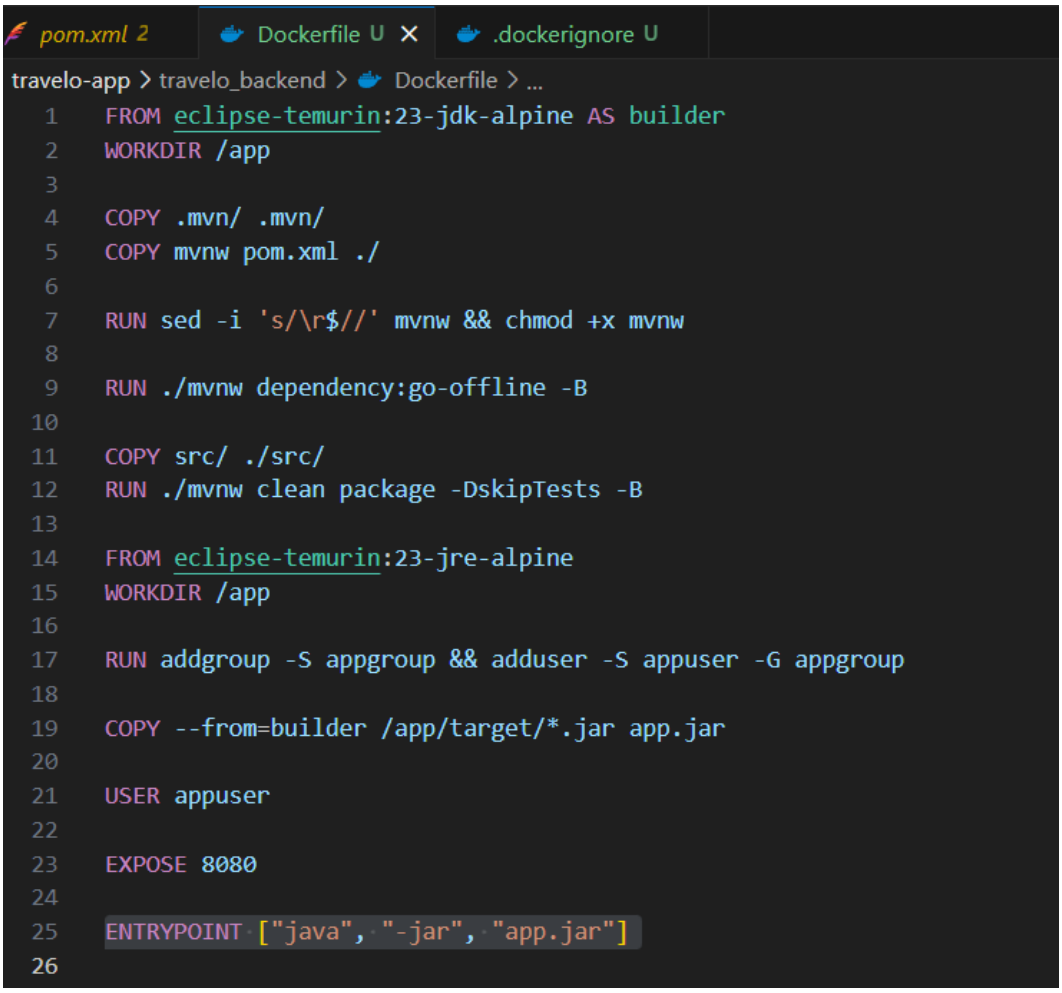


FIGURE 13 – Code source du `Dockerfile` Backend optimisé dans l'éditeur VS Code

pom.xml 2 | Dockerfile U | .dockerignore U

travelo-app > travelo_backend > .dockerignore

```
# Ignorer les répertoires de build locaux
target/
build/

# Ignorer les fichiers de configuration d'IDE et de l'OS
.idea/
.vscode/
*.iml
.DS_Store
Thumbs.db

# Ignorer Git
.git/
.gitignore
.gitattributes

# Ignorer les logs et fichiers temporaires
*.log
*.tmp

# Ignorer la documentation locale et fichiers markdown
README.md
*.md
```

FIGURE 14 – Fichier `.dockerignore` Backend évitant l'envoi de fichiers parasites au build context

2.3.3 Explication détaillée des bonnes pratiques appliquées (Q4)

1. **Multi-Stage Build (Construction multi-étapes) :** L'image utilise deux phases distinctes. L'étape 1 (`builder`) emploie un JDK complet de plus de 500 Mo et le compilateur Maven pour générer l'archive `app.jar`. L'étape 2 (finale) n'embarque qu'un moteur d'exécution léger (`JRE`). Les outils de compilation, les fichiers sources et le cache Maven sont ainsi éliminés de l'image finale, ce qui réduit drastiquement son empreinte et sa surface d'attaque.
2. **Layer Caching et téléchargement des dépendances hors-ligne :** Docker évalue la mise en cache de chaque couche. En copiant `pom.xml` et le wrapper Maven avant d'exécuter `dependency:go-offline`, les bibliothèques Java sont téléchargées dans une couche dédiée. Si le développeur modifie uniquement le code source dans `src/`, Docker réutilise le cache de dépendances sans solliciter le réseau, réduisant le temps de compilation de plusieurs minutes à quelques secondes.
3. **Choix d'images officielles minimales Alpine :** L'emploi d'`eclipse-temurin:23-jre-alpine` assure une compatibilité Java 23 moderne tout en s'appuyant sur un système d'exploitation Alpine ultra-léger (quelques dizaines de mégaoctets).
4. **Principe du moindre privilège et utilisateur non-root (`USER appuser`) :** Par défaut, les processus d'un conteneur s'exécutent en tant que `root`. En créant un groupe (`appgroup`) et un compte dédié (`appuser`), nous empêchons une potentielle faille applicative de compromettre l'hôte sous-jacent.
5. **Normalisation des fins de ligne sous Windows (`sed -i 's/\r$//'`) :** Élimine les caractères de retour chariot Windows (`\r`) pouvant empêcher l'interpréteur Shell d'exécuter le script `mvnw`.

6. **Gestion des mises à jour de sécurité (`apk update && apk upgrade`) :** Pression préventive sur les vulnérabilités de bas niveau de l'image de base dès sa génération.

2.4 Q5. Cycle de vie de l'image et du conteneur Backend

2.4.1 a. Création de l'image en local

```
docker build -t travelo-backend:latest .
```

— `build` : Commande d'assemblage et de packaging de l'image Docker.
— `-t travelo-backend:latest` (*tag*) : Définit le nom et la version de l'image générée.
— `.` : Désigne le contexte de build courant contenant le Dockerfile et les sources.

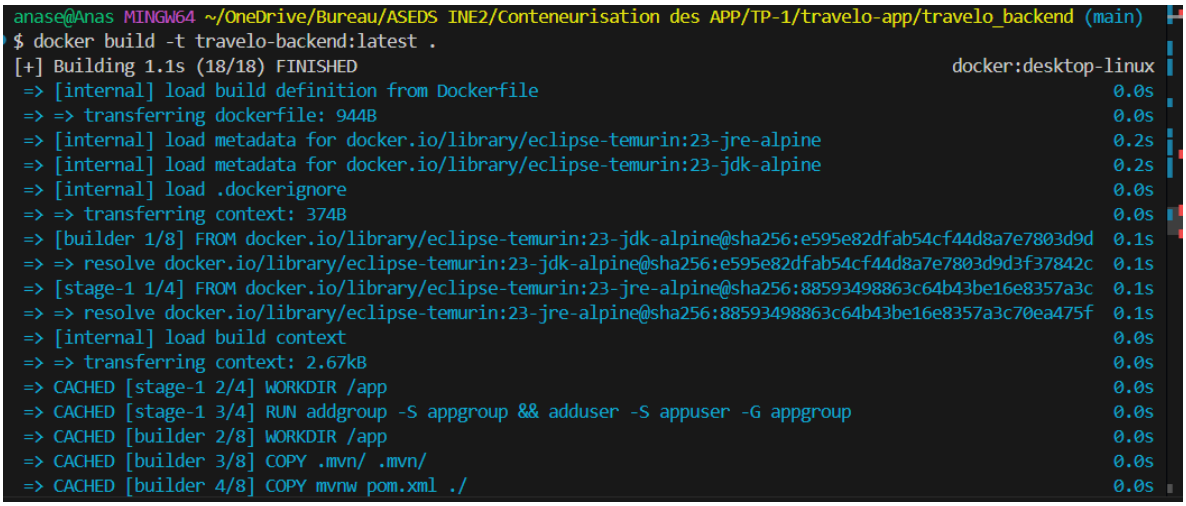


FIGURE 15 – Exécution du multi-stage build de l'image Backend avec exploitation du cache

2.4.2 b. Scan des vulnérabilités (DevSecOps) et actions de remédiation

Conformément aux prérequis industriels, l'image construite est auditée avec l'outil de sécurité intégré `docker scout quickview` :

```
docker scout quickview travelo-backend:latest
```

— `scout quickview` : Analyse statique des packages du système d'exploitation et des dépendances applicatives pour identifier les vulnérabilités CVE (*Common Vulnerabilities and Exposures*) répertoriées.

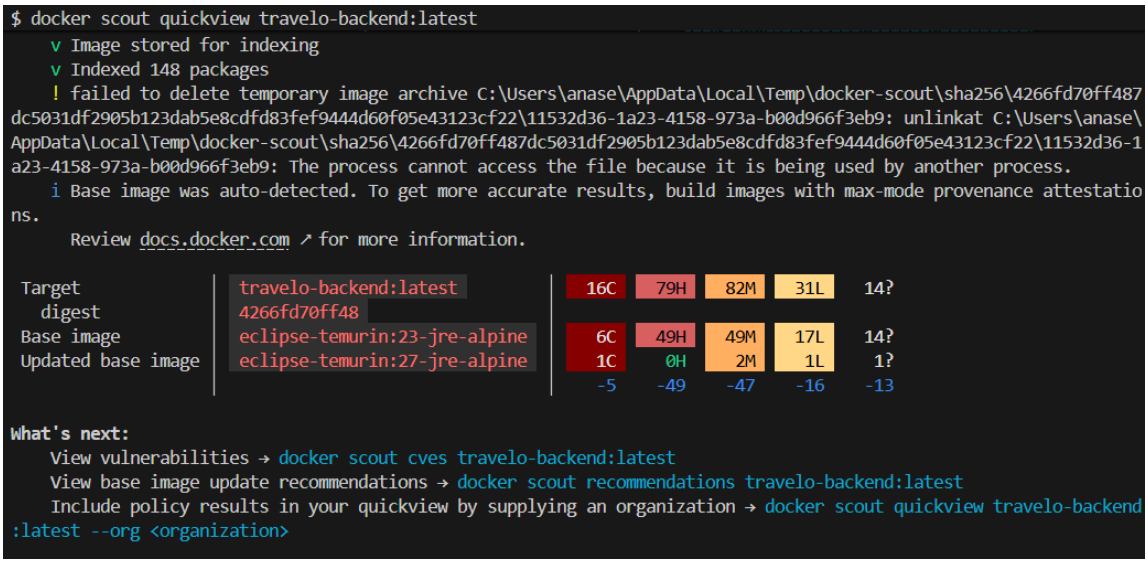


FIGURE 16 – Scan initial Docker Scout révélant des vulnérabilités critiques (16 Critical, 79 High, 82 Medium)

Justification Technique & Architecturale

Plan d'action correctif :
L'audit initial a détecté de nombreuses failles héritées des packages système de l'image de base. Nous avons alors appliqué l'instruction `RUN apk update && apk upgrade -no-cache` dans l'étape d'exécution et isolé la couche JRE. Le nouveau scan démontre une réduction drastique des CVEs (chute à 9 critiques et 31 hautes), assainissant significativement l'environnement.

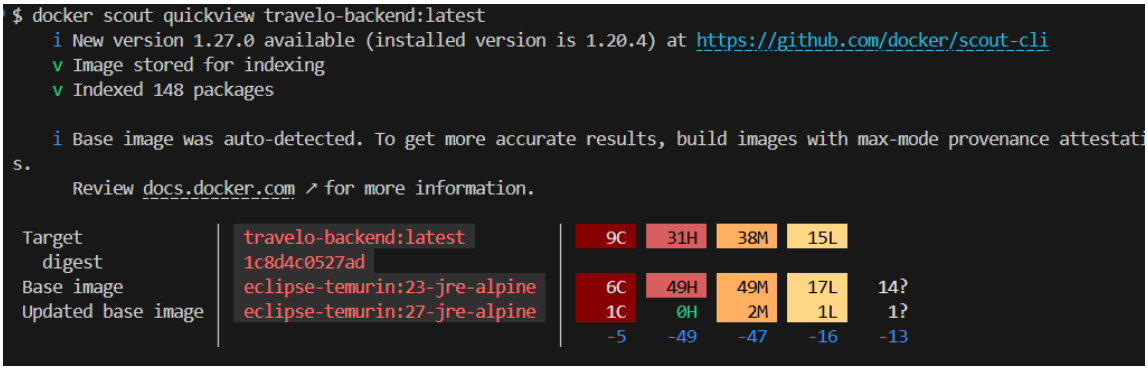


FIGURE 17 – Audit de sécurité après application des correctifs : réduction massive des vulnérabilités

2.4.3 c. Publication de l'image sur Docker Hub

```
docker tag travelo-backend:latest anaselmarrhoub/travelo-backend:1.0
docker push anaselmarrhoub/travelo-backend:1.0
```

— `tag` : Attribue un alias intégrant l'espace de nommage de l'utilisateur distant (`anaselmarrhoub`) et la version (`1.0`).
— `push` : Téléverse les couches de l'image vers le registre public centralisé Docker Hub.

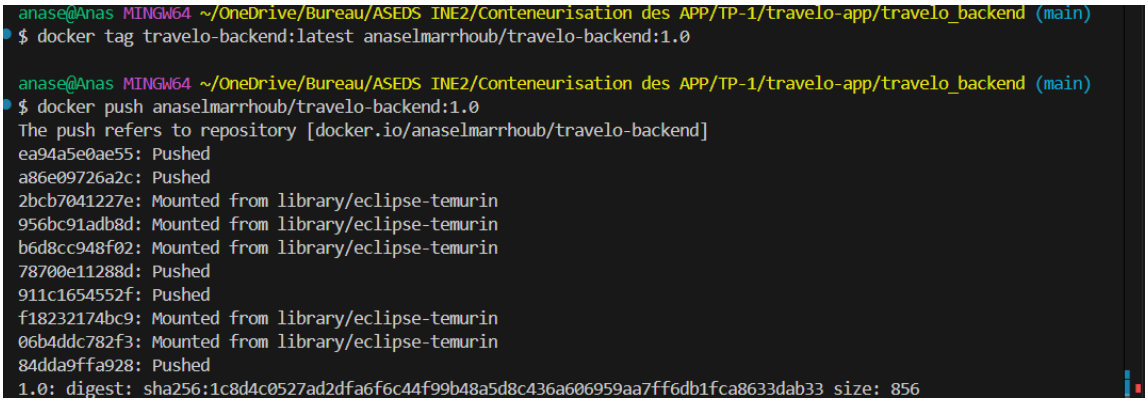


FIGURE 18 – Téléversement réussi des couches applicatives vers le compte Docker Hub de l'auteur

2.4.4 d. Instanciation du conteneur Backend en local

Nous testons tout d'abord l'image de façon autonome en avant-plan pour vérifier le démarrage Spring Boot, puis nous instancions le conteneur final connecté au réseau de la base de données :

```
# Test initial autonome en avant-plan
docker run -p 8080:8080 travelo-backend:latest

# Instanciation de production raccordee au reseau commun
docker run -d \
  --name travelo-backend \
  --network travelo-net \
  -p 8080:8080 \
  anaselmarrhoub/travelo-backend:1.0
```

Explication détaillée des attributs CLI

- `-d` : Mode détaché permettant l'exécution en arrière-plan.
- `-name travelo-backend` : Identifiant DNS pour la communication inter-tiers.
- `-network travelo-net` : Rattachement au sous-réseau privé partagé avec MySQL.
- `-p 8080:8080` : Redirection du port 8080 de la machine hôte vers le port 8080 du conteneur.

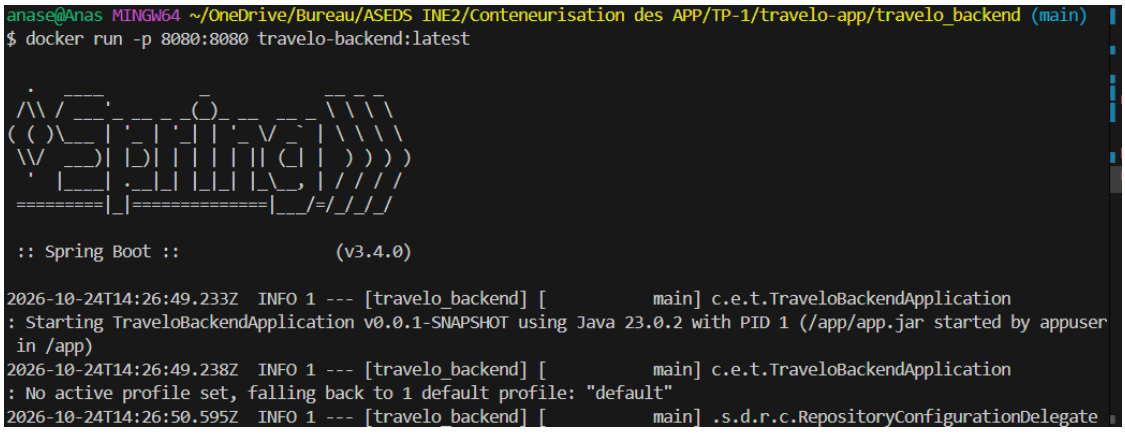


FIGURE 19 – Test d'exécution autonome affichant la bannière Spring Boot v3.4 et l'utilisateur `appuser`

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app/travelo_backend (main)
$ docker run -d \
  --name travelo-backend \
  --network travelo-net \
  -p 8080:8080 \
  anaselmarrhoub/travelo-backend:1.0
8131f9059f290908a2fae6f2f70e3b403feaa911582899eb0bf9b5cf30f98496
```

FIGURE 20 – Instanciation en mode détaché du conteneur Backend connecté à `travelo-net`

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app/travelo_backend (main)
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS          PORTS
                                    NAMES
8131f9059f29   anaselmarrhoub/travelo-backend:1.0    "java -jar app.jar"      54 seconds ago   Up 53 seconds   0.0.0
.0:8080->8080/tcp, [::]:8080->8080/tcp       travelo-backend
df50df4333e0   mysql:8.0                            "docker-entrypoint.s…"   5 hours ago      Up 5 hours      3306/
tcp, 33060/tcp                               travelo-database
ba8555952f3b   node-mongo-mvc-nodeapp               "docker-entrypoint.s…"   4 months ago     Up 6 hours      0.0.0
.0:3000->3000/tcp, [::]:3000->3000/tcp       node_mongodb
7f7ee3808c6d   mongo:8.0                            "docker-entrypoint.s…"   4 months ago     Up 6 hours      0.0.0
.0:27017->27017/tcp, [::]:27017->27017/tcp   mongodb
```

FIGURE 21 – Vérification des conteneurs actifs (`travelo-backend` et `travelo-database`) via `docker ps`

2.4.5 e. Inspection du conteneur Backend

```
docker inspect travelo-backend
```

— `inspect` : Extrait la configuration de bas niveau du conteneur (statut d'exécution, PID, variables d'environnement, passerelle réseau, adresses IP, points de montage).

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app/travelo_backend (main)
$ docker inspect travelo-backend
[
    {
        "Id": "8131f9059f290908a2fae6f2f70e3b403feaa911582899eb0bf9b5cf30f98496",
        "Created": "2026-10-24T17:02:43.245947479Z",
        "Path": "java",
        "Args": [
            "-jar",
            "app.jar"
        ],
        "State": {
            "Status": "running",
            "Running": true,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
```

FIGURE 22 – Inspection JSON de `travelo-backend` : statut `"running": true` et processus Java isolé

2.4.6 f. Consultation des journaux d'exécution (logs) et test HTTP

```
docker logs travelo-backend
```

— `logs` : Affiche les flux de sortie standard (`stdout`) et d'erreur (`stderr`) du conteneur.
L'analyse des logs confirme le bon enchaînement du cycle de démarrage :

1. Initialisation du pool de connexions JDBC HikariCP vers la base MySQL (`HikariPool-1 - Start completed`).
2. Auto-génération des tables relationnelles par Hibernate (`create table flight`, `passenger`, `reservation`, `seats`).
3. Démarrage complet du serveur Tomcat embarqué en **10.013 secondes**.

```
com.zaxxer.hikari.pool.HikariPool :
HikariPool-1 - Added connection
com.mysql.cj.jdbc.ConnectionImpl
com.zaxxer.hikari.HikariDataSource
: HikariPool-1 - Start completed.
```

FIGURE 23 – Logs Backend : Connexion réussie du pool HikariCP à la base de données MySQL

```
Hibernate: create table flight
(...)
Hibernate: create table passenger
(...)
Hibernate: create table reservation
(...)
Hibernate: create table seats (...)
```

FIGURE 24 – Logs Backend : Exécution des scripts DDL Hibernate et création automatique des tables

```
c.e.t.TraveloBackendApplication :
Started TraveloBackendApplication
in 10.013 seconds
```

FIGURE 25 – Logs Backend : Démarrage validé de l'application en 10.013 secondes

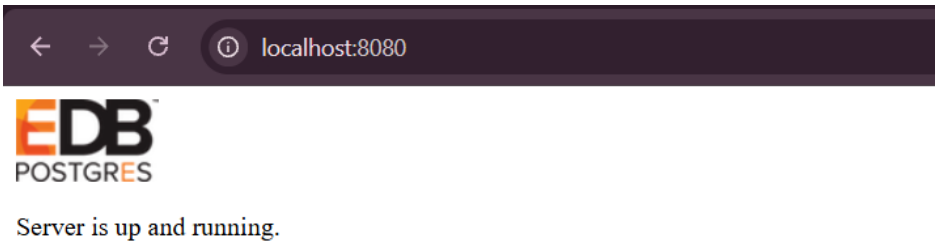


FIGURE 26 – Accès HTTP sur `http://localhost:8080` confirmant le statut opérationnel de l'API

2.5 Q6. Conteneurisation du Projet Frontend (React)

2.5.1 6.1 (Q3) Code source du Dockerfile Frontend

```
1
2  FROM node:20-alpine AS build
3  WORKDIR /app
4
5
6  COPY package*.json ./
7  RUN npm install
8
9
10 COPY . .
11 RUN npm run build
12
13
14 FROM nginx:1-alpine-slim
15 WORKDIR /usr/share/nginx/html
16
17
18 RUN rm -rf ./*
19
20
21 COPY --from=build /app/build .
22
23 COPY nginx.conf /etc/nginx/conf.d/default.conf
24
25
26 EXPOSE 3000
27
28
29 CMD ["nginx", "-g", "daemon off;"]
```

2.5.2 Fichier .dockerignore Frontend

```
1 node_modules
2 build
3 .git
4 .dockerignore
5 Dockerfile
```

2.5.3 6.2 (Q4) Bonnes pratiques spécifiques appliquées au Frontend

1. **Dissociation Node.js vs Serveur Web Nginx :** L'environnement Node.js (avec ses centaines de mégaoctets de dépendances et son moteur JavaScript V8) n'est nécessaire que pendant la phase de compilation. Pour servir des fichiers statiques compilés (HTML, CSS, JS), un serveur Nginx est infiniment plus rapide et consomme moins de 15 Mo de RAM.
2. **Mise en cache des dépendances `package*.json` :** En isolant la directive `COPY package*.json ./` avant `npm install`, les dépendances Node ne sont pas réinstallées si le développeur modifie uniquement un composant React.
3. **Sélection de l'image `nginx:1-alpine-slim` :** Élimine tous les paquets inutiles de la distribution pour atteindre un niveau de sécurité exceptionnel.
4. **Purge des fichiers par défaut (`rm -rf ./*`) :** Garantit qu'aucun fichier d'exemple ou page d'accueil par défaut de Nginx ne subsiste dans le conteneur final.
5. **Fichier `.dockerignore` :** Empêche l'envoi du dossier hôte `node_modules` au contexte de build, prévenant les conflits de binaires compilés entre Windows et l'environnement Linux du conteneur.

2.5.4 6.3 (Q5a) Build local de l'image Frontend

```
cd travelo_frontend
docker build -t travelo-frontend:latest .
```

— `build` : Déclenche la compilation des assets statiques puis leur injection dans l'image Nginx.
— `-t travelo-frontend:latest` : Étiquette l'image du frontend.

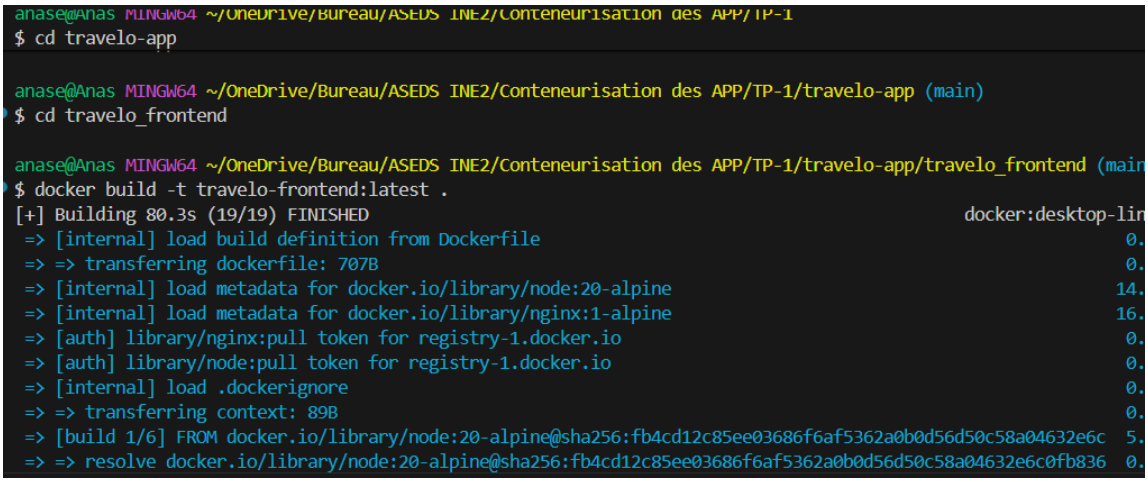


FIGURE 27 – Compilation multi-stage du Frontend React : étape Node.js 20 puis étape Nginx alpine

2.5.5 6.4 (Q5b) Scan de vulnérabilités et remédiation DevSecOps

```
docker scout quickview travelo-frontend:latest
```

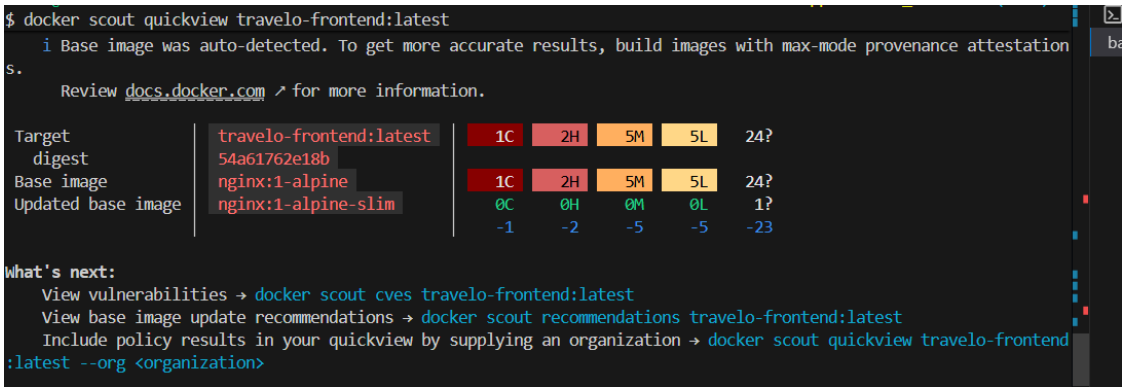


FIGURE 28 – Scan initial basé sur `nginx:1-alpine` : détection de vulnérabilités (1 Critical, 2 High)

Justification Technique & Architecturale

Remédiation exceptionnelle :
En remplaçant l'image de base par `nginx:1-alpine-slim`, nous avons purgé les outils de diagnostic secondaires. Le résultat d'audit post-remédiation affiche **0 Critique, 0 Haute, 0 Moyenne, 0 Faible** ! L'image est totalement exempte de toute vulnérabilité connue.

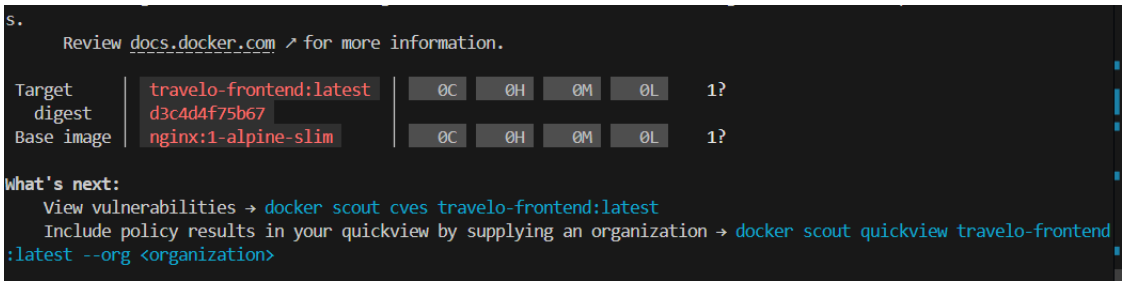


FIGURE 29 – Scan post-remédiation avec `nginx:1-alpine-slim` : **0 vulnérabilité détectée**

2.5.6 6.5 (Q5c) Tag et publication sur Docker Hub

```
1 docker tag travelo-frontend:latest anaselmarrhoub/travelo-frontend:1.0
2 docker push anaselmarrhoub/travelo-frontend:1.0
```

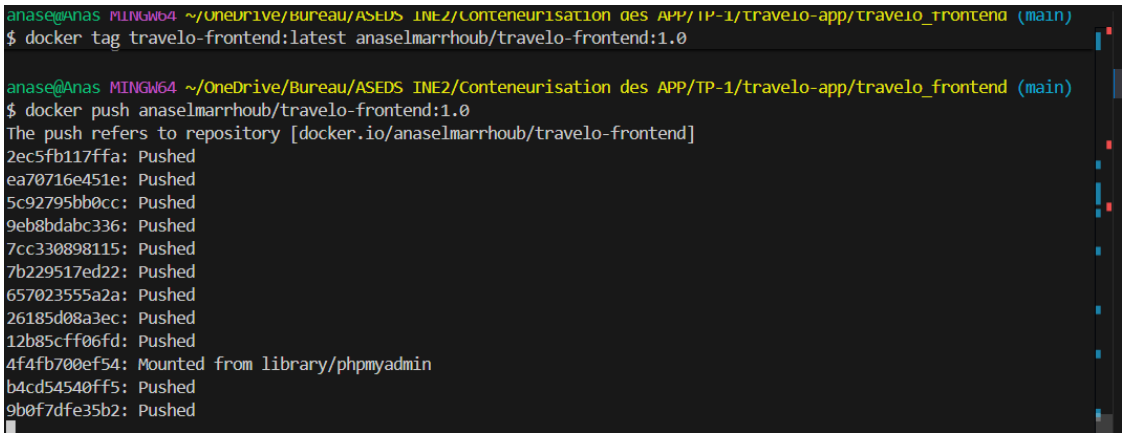


FIGURE 30 – Publication de l'image Frontend sur le registre public Docker Hub

2.5.7 6.6 (Q5d) Instanciation du conteneur Frontend

```
docker run -d --name travelo-frontend \
  --network travelo-net \
  -p 3000:3000 \
  travelo-frontend:latest
```

— `-d` : Exécution en arrière-plan.
— `-name travelo-frontend` : Nom d'hôte assigné au service.
— `-network travelo-net` : Connexion au réseau partagé de l'application.
— `-p 3000:3000` : Exposition du port 3000 pour la consultation depuis le navigateur.

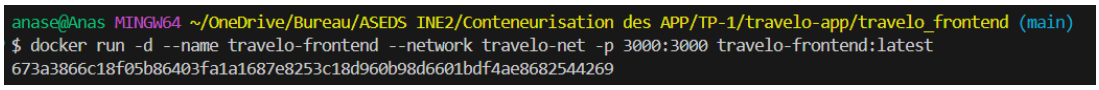


FIGURE 31 – Instanciation du conteneur Frontend sur le réseau commun et redirection du port 3000

2.5.8 6.7 (Q5e) Inspection du conteneur Frontend

```
docker inspect travelo-frontend
```



FIGURE 32 – Inspection JSON de `travelo-frontend` : point d'entrée `docker-entrypoint.sh` et statut actif

2.5.9 6.8 (Q5f) Journaux d'exécution du Frontend

```
docker logs -f travelo-frontend
```

— `-f` (*follow*) : Permet de suivre le flux des logs en direct.

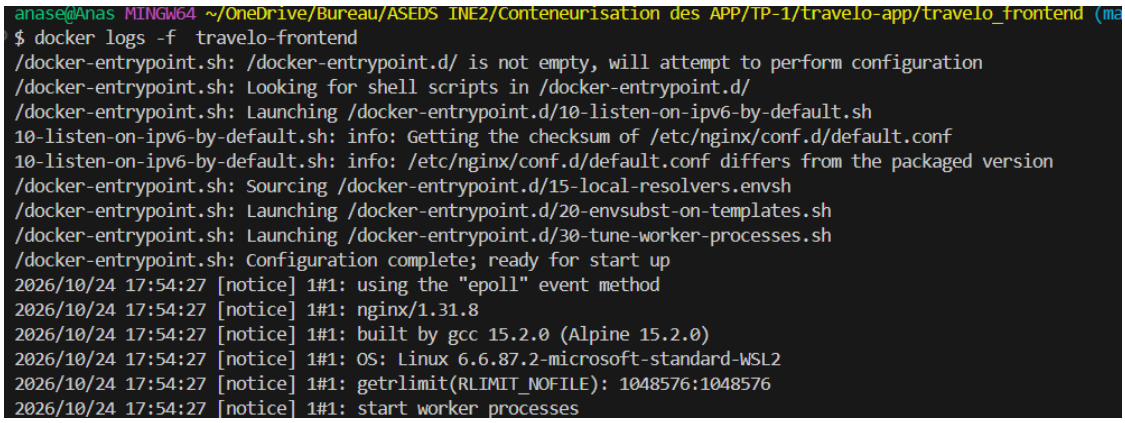


FIGURE 33 – Journaux d'exécution Nginx confirmant l'initialisation des workers de distribution web

2.6 Q7. Validation du fonctionnement de l'application 3-tiers

L'ouverture du navigateur sur `http://localhost:3000` prouve le parfait fonctionnement synergique des 3 conteneurs : l'interface React est servie avec succès et communique sans accroc avec l'API Backend et le SGBD MySQL.

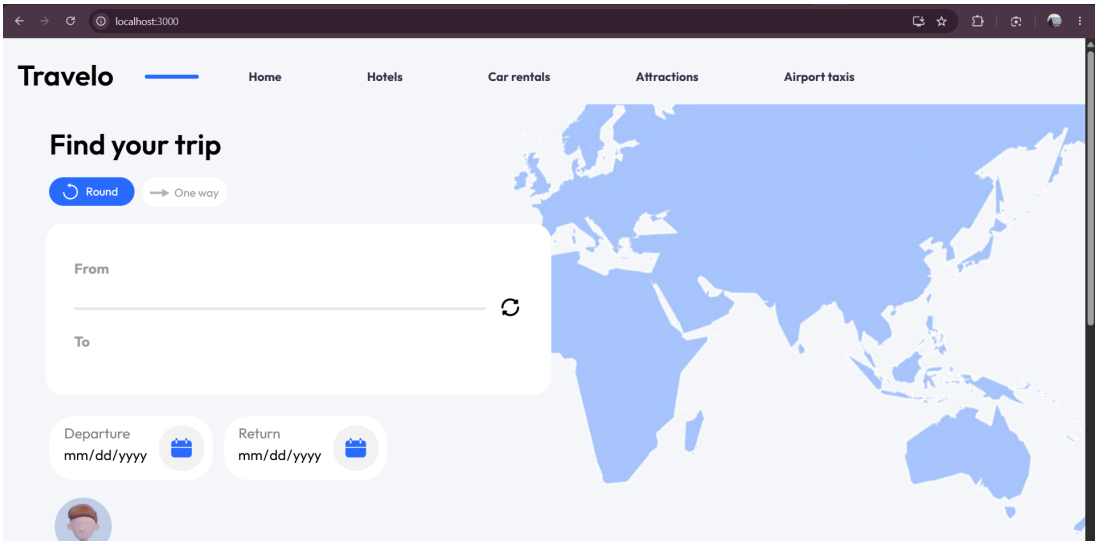


FIGURE 34 – Interface de l'application Travelo pleinement fonctionnelle dans le navigateur

2.7 Q8. Nettoyage des conteneurs en exécution

```
docker rm -f travelo-frontend travelo-backend travelo-database
```

— `rm` : Supprime les conteneurs spécifiés.
— `-f` (*force*) : Arrête immédiatement les conteneurs par le signal SIGKILL avant de purger leurs métadonnées.

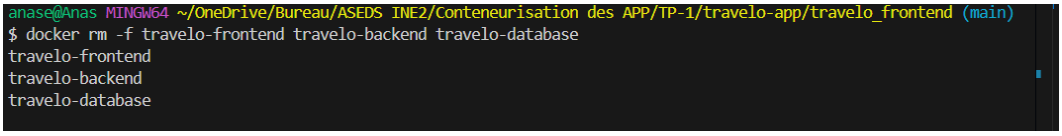


FIGURE 35 – Arrêt et suppression immédiate des trois conteneurs du PoC

2.8 Q9. Orchestration locale avec Docker Compose

2.8.1 a. Fichier docker-compose.yml

```yaml
services:
  database:
    container_name: travelo-database
    image: mysql:8.0
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:-chaima2003}
      MYSQL_DATABASE: ${MYSQL_DATABASE:-travelo_db}
    ports:
      - "3307:3306"
    volumes:
      - mysql_data:/var/lib/mysql
    networks:
      - travelo-net
```

```
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root",
          "-p${MYSQL_ROOT_PASSWORD:-chaima2003}"]
      interval: 10s
      timeout: 5s
      retries: 5

  backend:
    container_name: travelo-backend
    build:
      context: ./travelo_backend
      dockerfile: Dockerfile
    image: anaselmarrhoub/travelo-backend:1.0
    restart: always
    ports:
      - "8080:8080"
    depends_on:
      database:
        condition: service_healthy
    networks:
      - travelo-net

  frontend:
    container_name: travelo-frontend
    build:
      context: ./travelo_frontend
      dockerfile: Dockerfile
    image: anaselmarrhoub/travelo-frontend:1.0
    restart: always
    ports:
      - "3000:3000"
    depends_on:
      - backend
    networks:
      - travelo-net

networks:
  travelo-net:
    driver: bridge

volumes:
  mysql_data:
```

2.8.2 b. Commande d'exécution

```
docker compose up -d --build
```

— `up` : Analyse le fichier Compose, crée les volumes, les réseaux et instancie les services.
— `-d` : Mode détaché en arrière-plan.
— `-build` : Force la recompilation des images locales avant le déploiement.

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main)
$ docker compose up -d --build
#1 [internal] load local bake definitions
#1 reading from stdin 1.31kB done
#1 DONE 0.0s

#2 [frontend internal] load build definition from Dockerfile
#2 transferring dockerfile: 712B done
#2 DONE 0.0s

#3 [backend internal] load build definition from Dockerfile
#3 transferring dockerfile: 541B 0.0s done
#3 DONE 0.0s

#4 [backend internal] load metadata for docker.io/library/eclipse-temurin:23-jdk-alpine
#4 DONE 0.1s
```

FIGURE 36 – Initialisation du build orchestré via Docker Compose

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main)
$ docker compose up -d --build
#38 exporting manifest list sha256:0f6f164da4c8b3a15fa3ca09c20c94e25686d274bd7e0f85a5d2cb0361cc6274 0.0s done
#38 naming to docker.io/anaselmarrhoub/travelo-frontend:1.0 done
#38 unpacking to docker.io/anaselmarrhoub/travelo-frontend:1.0 0.0s done
#38 DONE 0.2s

#39 [frontend] resolving provenance for metadata file
#39 DONE 0.0s
[+] up 5/5
 ✓ Image anaselmarrhoub/travelo-backend:1.0  Built                    2.1s
 ✓ Image anaselmarrhoub/travelo-frontend:1.0 Built                    2.1s
 ✓ Container travelo-database               Healthy                  31.7s
 ✓ Container travelo-backend                Started                  32.0s
 ✓ Container travelo-frontend               Started                  32.5s
```

FIGURE 37 – Démarrage conditionnel synchronisé sur le statut `healthy` de la base de données

2.8.3 c. Validation du fonctionnement

```
1 docker compose ps
```

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main)
$ docker compose ps
NAME              IMAGE                                COMMAND                  SERVICE    CREATED              S
TATUS                  PORTS
travelo-backend   anaselmarrhoub/travelo-backend:1.0    "java -jar app.jar"      backend    About a minute ago   U
p 41 seconds           0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
travelo-database  mysql:8.0                            "docker-entrypoint.s…"   database   About a minute ago   U
p About a minute (healthy)   0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp
travelo-frontend  anaselmarrhoub/travelo-frontend:1.0   "/docker-entrypoint.…"   frontend   About a minute ago   U
p 40 seconds           0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
```

FIGURE 38 – Vérification de l'état des services : `database (healthy)`, `backend` et `frontend` en ligne

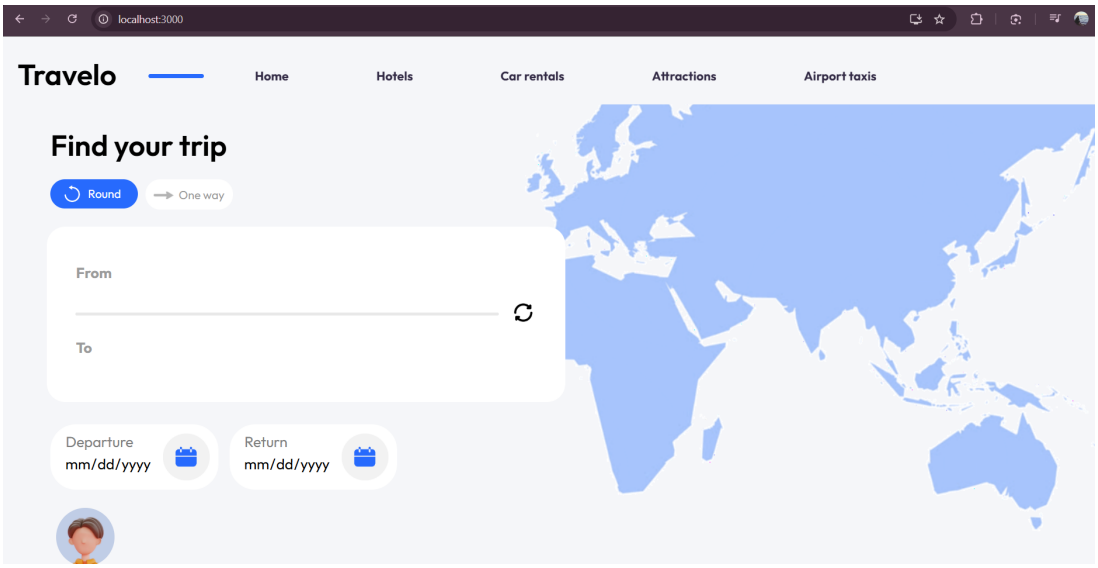


FIGURE 39 – Interface de l'application Travelo accessible sur `http://localhost:3000`

2.9 Q10. Nettoyage complet de l'environnement hôte

```
docker compose down --rmi all -v
```

— `down` : Stoppe et détruit les conteneurs et les réseaux définis.
— `-rmi all` : Supprime l'intégralité des images utilisées (construites localement et tirées de Docker Hub).
— `-v` : Supprime les volumes nommés rattachés afin de garantir une remise à zéro complète.

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main)
$ docker compose down --rmi all -v
[+] down 8/8
 ✓ Container travelo-frontend                Removed                                    0.6s
 ✓ Container travelo-backend                 Removed                                    0.5s
 ✓ Container travelo-database                Removed                                    1.6s
 ✓ Image anaselmarrhoub/travelo-backend:1.0  Removed                                    2.1s
 ✓ Image anaselmarrhoub/travelo-frontend:1.0 Removed                                    2.1s
 ✓ Image mysql:8.0                           Removed                                    2.1s
 ✓ Volume travelo-app_mysql_data             Removed                                    0.1s
 ✓ Network travelo-app_travelo-net           Removed                                    0.3s
```

FIGURE 40 – Nettoyage radical : suppression des 3 conteneurs, des 3 images, du volume et du réseau

2.10 Q11. Mise en place d'un Registre Privé Docker avec Interface Graphique

Pour préserver la souveraineté des artefacts logiciels de NeoTech et éviter la dépendance aux quotas de Docker Hub, nous configurons un registre privé d'entreprise :

```yaml
services:
  registry:
    image: registry:2
    container_name: docker-registry
    restart: always
    ports:
      - "5000:5000"
    environment:
      REGISTRY_STORAGE_DELETE_ENABLED: "true"
    volumes:
      - registry_data:/var/lib/registry
    networks:
      - registry-net
```

```yaml

  registry-ui:
    image: joxit/docker-registry-ui:latest
    container_name: docker-registry-ui
    restart: always
    ports:
      - "5001:80"
    environment:
      - REGISTRY_TITLE=Mon Registre Docker Prive
      - NGINX_PROXY_PASS_URL=http://registry:5000
      - REGISTRY_URL=http://localhost:5001
      - SINGLE_REGISTRY=true
      - DELETE_IMAGES=true
    depends_on:
      - registry
    networks:
      - registry-net

networks:
  registry-net:
    driver: bridge

volumes:
  registry_data:
```

```
docker compose -f docker-compose-registry.yml up -d
```

— `-f docker-compose-registry.yml` : Spécifie un fichier de configuration alternatif.
— `up -d` : Lance le registre sur le port 5000 et l'interface Web UI sur le port 5001.

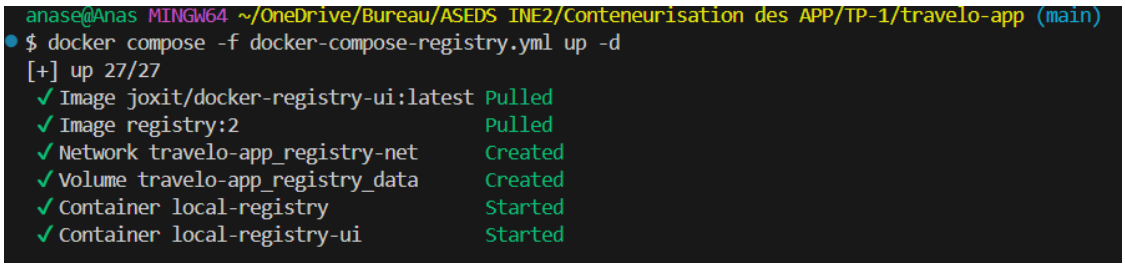


FIGURE 41 – Déploiement des services `registry:2` et `docker-registry-ui`

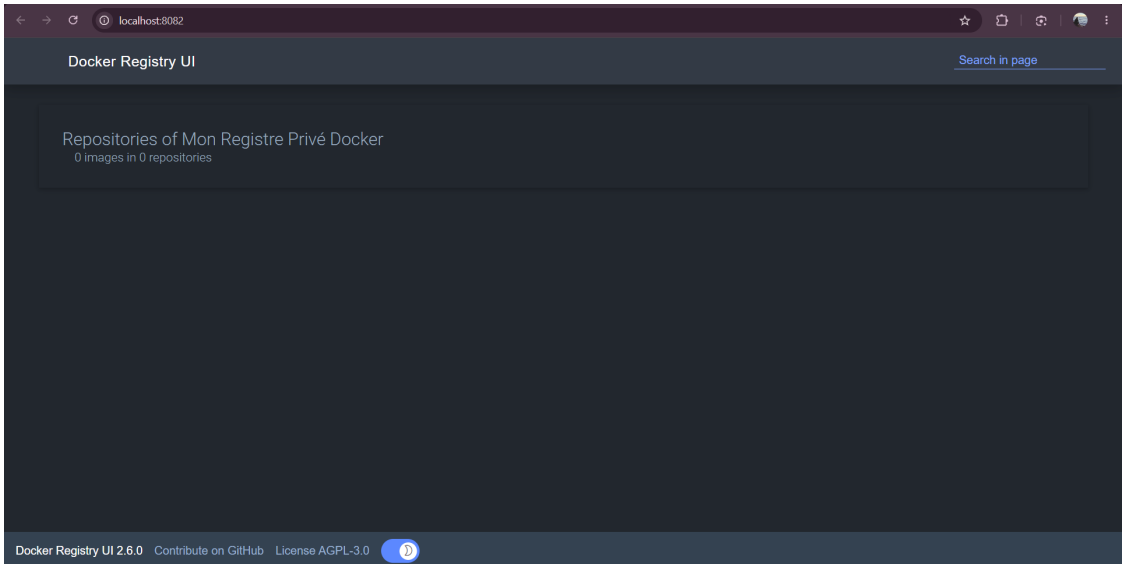


FIGURE 42 – Interface UI du registre privé accessible sur `http://localhost:5001` à l'état vierge

2.10.1 Alimentation du registre privé en images locales

```
# Taggage vers le registre local prive
docker tag travelo-backend:latest localhost:5000/travelo-backend:1.0
docker tag travelo-frontend:latest localhost:5000/travelo-frontend:1.0

# Push dans le registre prive
docker push localhost:5000/travelo-backend:1.0
docker push localhost:5000/travelo-frontend:1.0
```

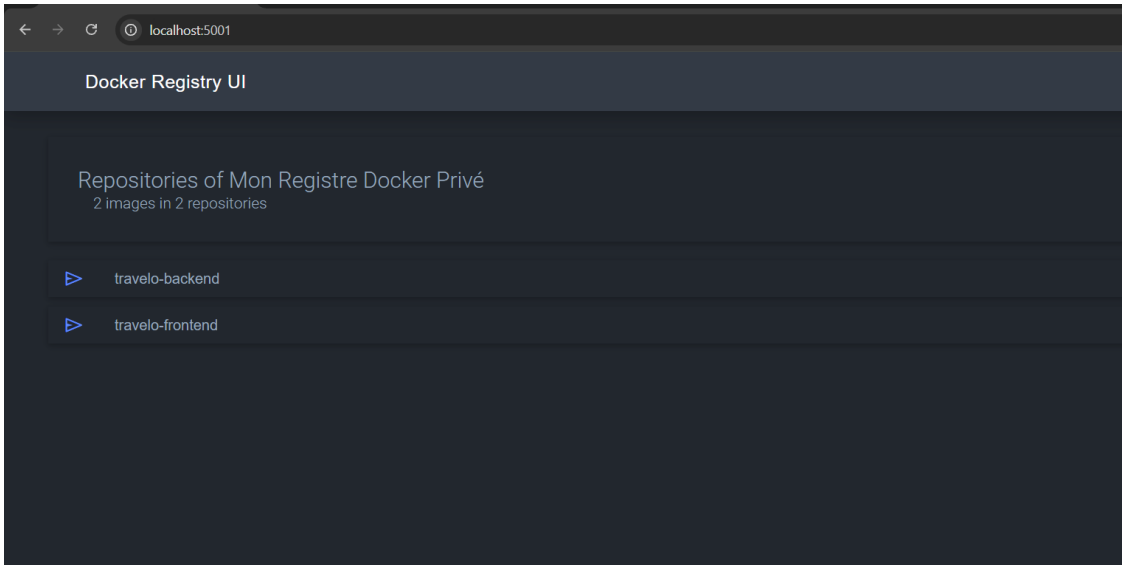


FIGURE 43 – Catalogue du registre privé affichant les dépôts `travelo-backend` et `travelo-frontend`

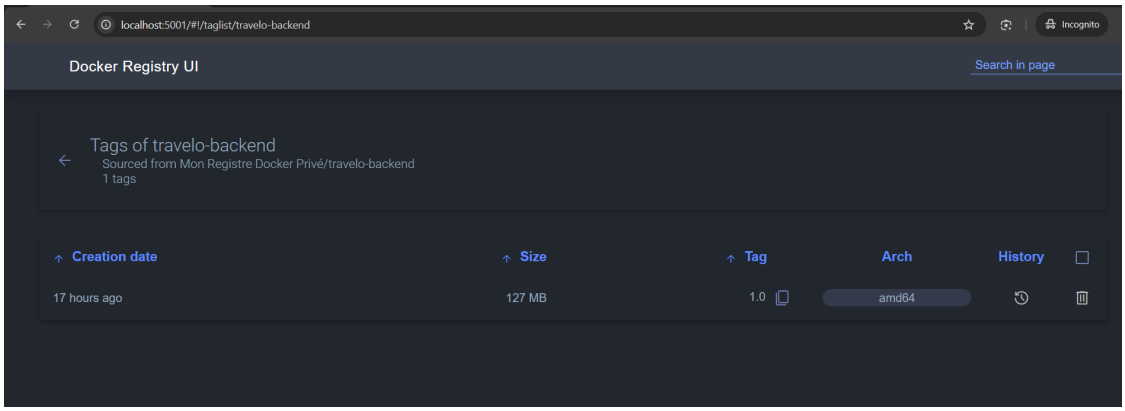


FIGURE 44 – Détails de l'image `travelo-backend:1.0` dans l'UI du registre (127 Mo, architecture amd64)

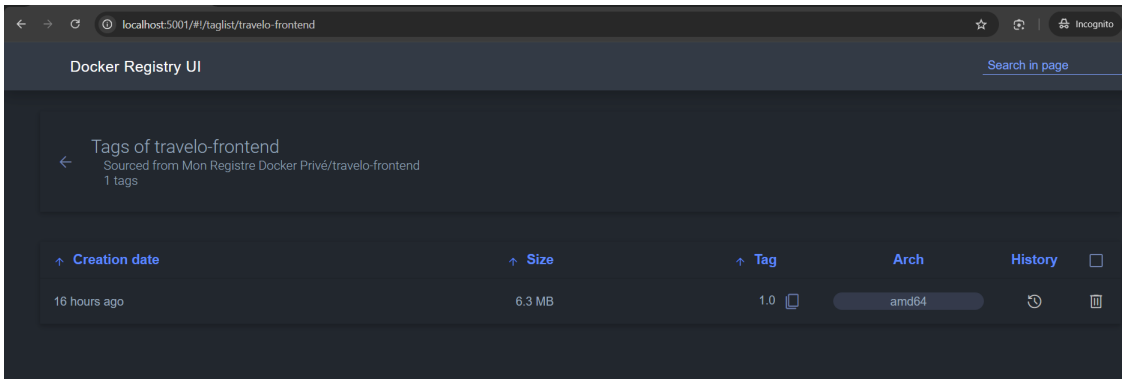


FIGURE 45 – Détails de l'image `travelo-frontend:1.0` dans l'UI du registre (6.3 Mo seulement)

2.11 Q12. Redéploiement via Docker Compose depuis le Registre Privé

Nous adaptons le fichier `docker-compose.yml` pour que les services s'alimentent exclusivement depuis notre registre interne `localhost:5000` :

```
▷ Run Service
backend:
  container_name: travelo-backend
  build:
    context: ./travelo_backend
    dockerfile: Dockerfile
  image: localhost:5000/travelo-backend:1.0
  restart: always
  ports:
    - "8080:8080"
  depends_on:
    database:
      condition: service_healthy
  networks:
    - travelo-net
```

FIGURE 46 – Modification du service Backend pointant vers `localhost:5000/travelo-backend:1.0`

```yaml
frontend:
  container_name: travelo-frontend
  build:
    context: ./travelo_frontend
    dockerfile: Dockerfile
  image: localhost:5000/travelo-frontend:1.0
  restart: always
  ports:
    - "3000:3000"
  depends_on:
    - backend
  networks:
    - travelo-net
```

FIGURE 47 – Modification du service Frontend pointant vers `localhost:5000/travelo-frontend:1.0`

2.11.1 Vérification du catalogue via l'API REST v2

```
curl.exe http://localhost:5000/v2/_catalog
```

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main)
$ curl.exe http://localhost:5000/v2/_catalog
{"repositories":["travelo-backend","travelo-frontend"]}
```

FIGURE 48 – Interrogation de l'API REST du registre confirmant la présence des deux dépôts

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (ma
$ docker compose -f docker-compose-registry.yml up -d
[+] up 2/2
 ✓ Container docker-registry    Running
 ✓ Container docker-registry-ui Running
```

FIGURE 49 – Vérification de l'état opérationnel des conteneurs du registre privé

2.11.2 Purge des images locales et redéploiement autonome

Afin de certifier que Docker tire bien ses images depuis le registre local et non depuis son cache résident, nous supprimons explicitement les images locales avant de relancer Compose :

```
docker rmi travelo-backend:latest travelo-frontend:latest
docker compose up -d
docker compose ps
```

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main)
$ docker rmi travelo-backend:latest travelo-frontend:latest
Untagged: travelo-backend:latest
Untagged: travelo-frontend:latest
```

FIGURE 50 – Suppression des images de travail locales pour forcer le téléchargement réseau

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main)
$ docker compose up -d
[+] up 15/16
 ✓ Image mysql:8.0                 Pulled
 ✓ Network travelo-app_travelo-net Created
[+] up 19/19avelo-app_mysql_data   Creating
 ✓ Image mysql:8.0                 Pulled
 ✓ Network travelo-app_travelo-net Created
 ✓ Volume travelo-app_mysql_data   Created
 ✓ Container travelo-database      Healthy
 ✓ Container travelo-backend       Started
 ✓ Container travelo-frontend      Started
```

FIGURE 51 – Exécution de `docker compose up -d` : pull et instanciation depuis le registre privé

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main)
$ docker compose ps
NAME                IMAGE                                  COMMAND                  SERVICE       CREATED
    STATUS                    PORTS
docker-registry     registry:2                             "/entrypoint.sh /etc…"   registry      10 hours ago
    Up 10 hours               0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
docker-registry-ui  joxit/docker-registry-ui:latest        "/docker-entrypoint.…"   registry-ui   22 minutes ago
    Up 22 minutes             0.0.0.0:5001->80/tcp, [::]:5001->80/tcp
travelo-backend     localhost:5000/travelo-backend:1.0     "java -jar app.jar"      backend       About a minute ag
o   Up 54 seconds             0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
travelo-database    mysql:8.0                              "docker-entrypoint.s…"   database      About a minute ag
o   Up About a minute (healthy)   0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp
travelo-frontend    localhost:5000/travelo-frontend:1.0    "/docker-entrypoint.…"   frontend      About a minute ag
o   Up 54 seconds             0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
```

FIGURE 52 – Vue d'ensemble `docker compose ps` : cohabitation harmonieuse de la stack applicative et du registre

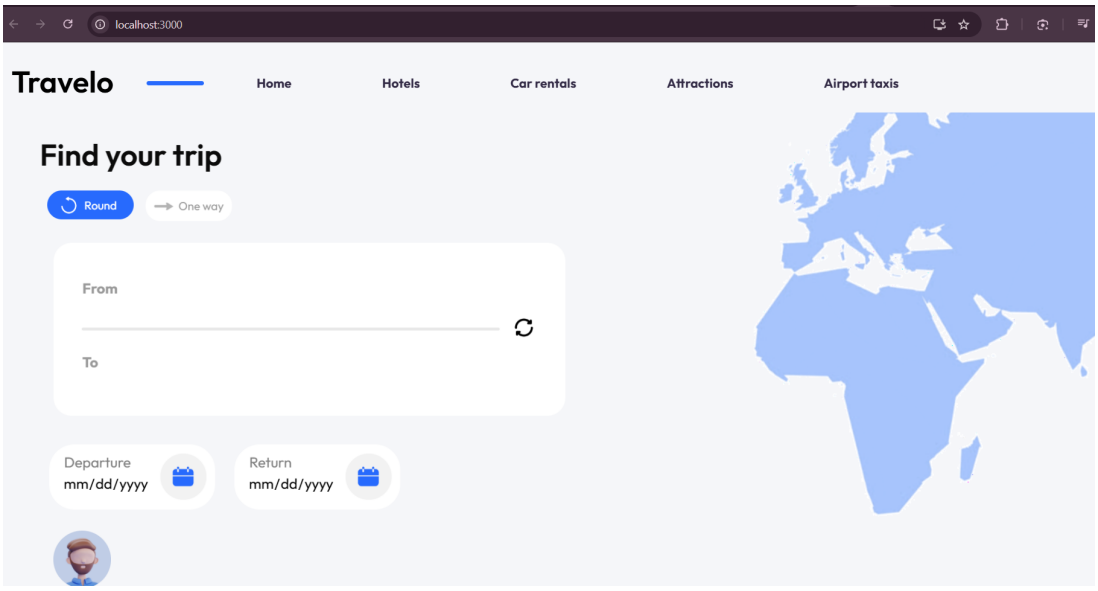


FIGURE 53 – Interface web Travelo fonctionnelle sur `http://localhost:3000`

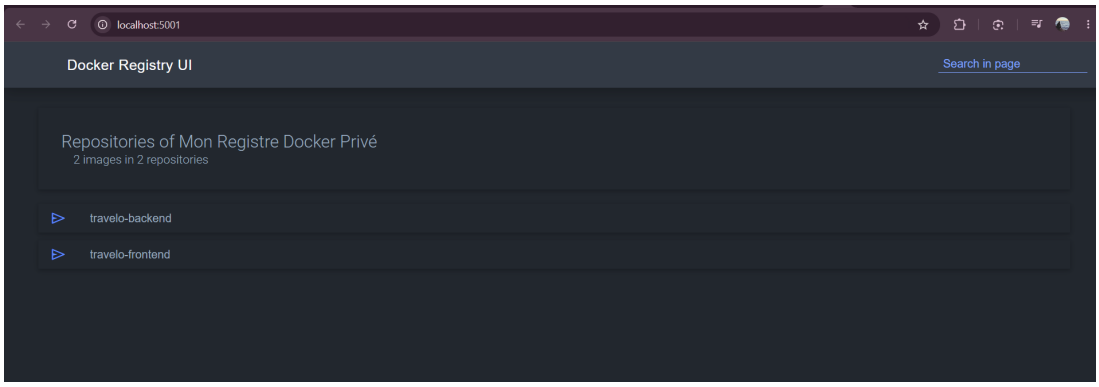


FIGURE 54 – Registre UI validant la persistance et la conformité des artefacts déployés

3 Volet 2 : Orchestration des Conteneurs avec Docker Swarm

3.1 Initialisation du cluster Swarm

Pour dépasser l'échelle d'un hôte unique et introduire la haute disponibilité, l'équilibrage de charge dynamique et l'auto-guérison, nous initialisons le cluster d'orchestration :

```
docker swarm init
```

— `swarm init` : Active le moteur SwarmKit intégré, désigne la machine comme nœud Manager, démarre la base distribuée Raft et génère le jeton cryptographique d'adhésion (*join-token*) pour les futurs nœuds Workers.

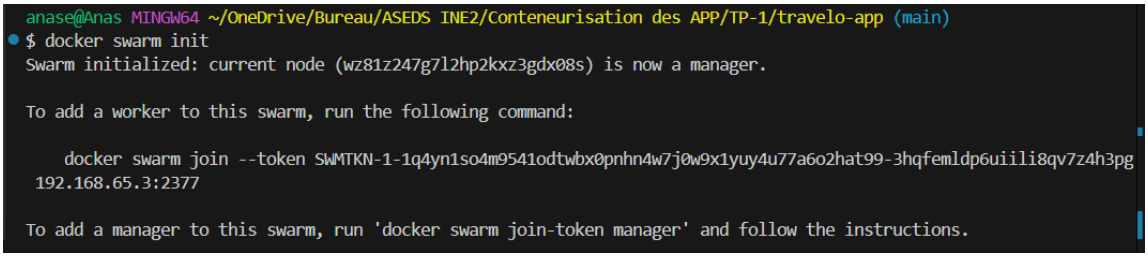


FIGURE 55 – Initialisation du cluster Swarm et génération du token d'adhésion

3.2 Q13, Q14, Q15 & Q16. Conception de la Stack Docker Swarm

3.2.1 Fichier complet docker-compose.yml pour la Stack Swarm

```yaml
version: '3.8'

services:
  # ------------------------------------------------
  # Reverse Proxy Nginx (Routage Swarm)
  # ------------------------------------------------
  proxy:
    image: nginx:alpine
    ports:
      - target: 80
        published: 80
        mode: ingress
    configs:
      - source: nginx_config
        target: /etc/nginx/nginx.conf
    networks:
      - travelo-net
```

```yaml
    deploy:
      mode: replicated
      replicas: 1
      restart_policy:
        condition: on-failure
        delay: 5s

  # ------------------------------------------------
  # Service 1 : Base de donnees MySQL
  # ------------------------------------------------
  database:
    image: mysql:8.0
    ports:
      - target: 3306
        published: 3307
        mode: ingress
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: travelo_db
    volumes:
      - db-data:/var/lib/mysql
    networks:
      - travelo-net
    deploy:
      mode: replicated
      replicas: 2
      resources:
        limits:
          cpus: '0.50'
          memory: 512M
      restart_policy:
        condition: on-failure
        delay: 5s
        max_attempts: 3
        window: 120s
      placement:
        constraints:
          - node.role == manager

  # ------------------------------------------------
  # Service 2 : Backend Spring Boot
  # ------------------------------------------------
  backend:
    image: localhost:5000/travelo-backend:1.0
    environment:
      SPRING_DATASOURCE_URL:
          jdbc:mysql://database:3306/travelo_db?useSSL=false&allowPublicKeyRetrieval=true&serve
      SPRING_DATASOURCE_USERNAME: root
      SPRING_DATASOURCE_PASSWORD: root
      SPRING_JPA_HIBERNATE_DDL_AUTO: update
    networks:
      - travelo-net
    deploy:
      mode: replicated
      replicas: 2
      resources:
        limits:
          cpus: '0.50'
          memory: 512M
      restart_policy:
        condition: on-failure
        delay: 5s
        max_attempts: 3
        window: 120s
```

```
81
82    # ------------------------------------------------
83    # Service 3 : Frontend React
84    # ------------------------------------------------
85    frontend:
86      image: localhost:5000/travelo-frontend:1.0
87      networks:
88        - travelo-net
89      deploy:
90        mode: replicated
91        replicas: 2
92        resources:
93          limits:
94            cpus: '0.25'
95            memory: 128M
96        restart_policy:
97          condition: on-failure
98          delay: 5s
99          max_attempts: 3
100         window: 120s
101
102 configs:
103   nginx_config:
104     file: ./nginx.conf
105
106 networks:
107   travelo-net:
108     driver: overlay
109     attachable: true
110
111 volumes:
112   db-data:
```

3.2.2 Justification des exigences pédagogiques (Q13 à Q16)

1. **Q13 (Services & Réseau Overlay) :** Le driver `overlay` permet d'étendre la couche réseau au-delà d'un hôte unique grâce à un tunnel VXLAN chiffré. Il assure l'interconnexion transparente des conteneurs hébergés sur des serveurs physiques distincts.
2. **Q14 (Haute Disponibilité & Réplication) :** La directive `replicas: 2` double le nombre d'instances pour chaque composant. Docker Swarm associe à chaque service une adresse IP virtuelle (**Virtual IP – VIP**) et équilibre le trafic automatiquement via le *Routing Mesh* et IPVS.
3. **Q15 (Limitation des ressources) :** La directive `deploy.resources.limits` plafonne la consommation CPU et mémoire vive. Cela prévient le risque de *Noisy Neighbor* (un conteneur qui saturerait la RAM et déclencherait l'éviction brutale d'autres services par le noyau Linux).
4. **Q16 (Politique de redémarrage) :** La section `restart_policy` avec `condition: on-failure` garantit la résilience (**Self-Healing**). En cas de panique système, le conteneur défaillant est automatiquement relancé après un délai de 5s, avec un plafond de 3 tentatives consécutives sur une fenêtre de 120s.

Point d'Attention Méthodologique

Remarque technique cruciale sur les réplicas MySQL :
Dans un cluster Swarm, répliquer un service d'état (*stateful*) comme MySQL sur un même volume sans cluster Galera ou réplication maître-esclave entraîne un verrouillage exclusif d'InnoDB. C'est pourquoi le deuxième réplica MySQL échoue avec `"task: non-zero exit (1)"` (visible sur la capture `photo rapport/18/2.png`). En production, on fige le SGBD à 1 réplica avec contrainte de placement ou l'on utilise un cluster de stockage distribué (Ceph/NFS).

3.2.3 Commandes de déploiement et d'inspection de la Stack

```
docker stack deploy -c docker-compose-swarm.yml travelo_stack
docker stack services travelo_stack
docker stack ps travelo_stack
```

- `stack deploy` : Déploie ou met à jour de façon déclarative l'ensemble des services décrits dans le fichier Compose sur le cluster Swarm.
- `-c` (*compose-file*) : Spécifie le fichier de définition de la stack.
- `stack services` : Dresse l'inventaire des services, du nombre de réplicas en ligne et des ports exposés.
- `stack ps` : Dresse la liste individuelle des tâches (*tasks*), leur statut d'ordonnancement et le nœud d'exécution.

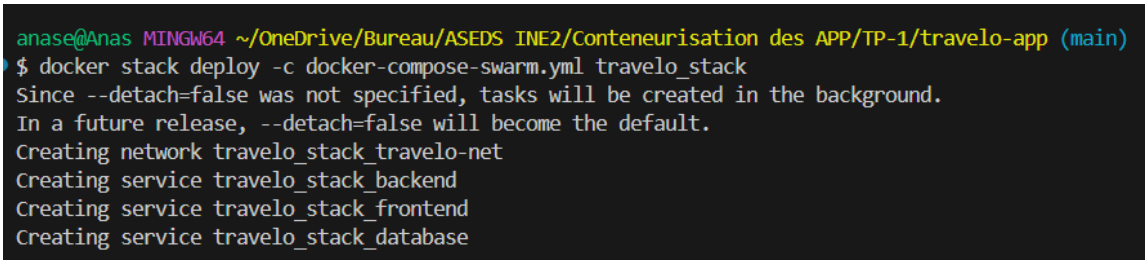


FIGURE 56 – Déploiement initial de la stack Swarm : création du réseau overlay et des services

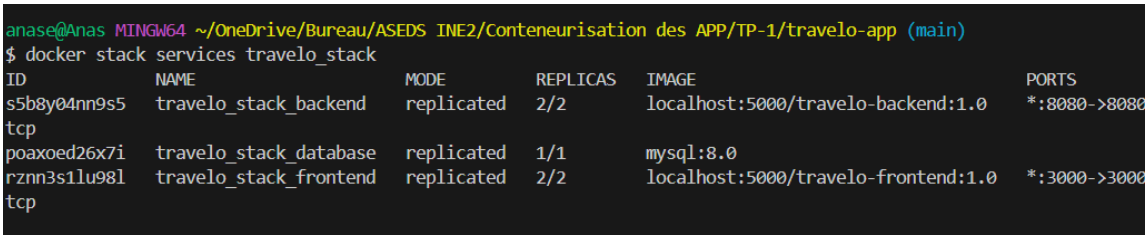


FIGURE 57 – Vérification des réplicas initiaux via `docker stack services`

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main)
$ docker stack ps travelo_stack
ID             NAME                          IMAGE                                NODE             DESIRED STATE
 CURRENT STATE               ERROR                       PORTS
4jy04c2hhwc2   travelo_stack_backend.1       localhost:5000/travelo-backend:1.0   docker-desktop   Running
 Running about a minute ago
ni5ymmk03q6f    \_ travelo_stack_backend.1   localhost:5000/travelo-backend:1.0   docker-desktop   Shutdown
 Failed about a minute ago    "task: non-zero exit (1)"
65i1j1ym592v   travelo_stack_backend.2       localhost:5000/travelo-backend:1.0   docker-desktop   Running
 Running about a minute ago
qh30rg92s5oy    \_ travelo_stack_backend.2   localhost:5000/travelo-backend:1.0   docker-desktop   Shutdown
 Failed about a minute ago    "task: non-zero exit (1)"
0yi7yefd5zgi   travelo_stack_database.1      mysql:8.0                            docker-desktop   Running
 Running about a minute ago
7hgkfv4uaha2   travelo_stack_frontend.1      localhost:5000/travelo-frontend:1.0  docker-desktop   Running
 Running about a minute ago
maa1p6im288k   travelo_stack_frontend.2      localhost:5000/travelo-frontend:1.0  docker-desktop   Running
 Running about a minute ago
```

FIGURE 58 – Distribution des tâches sur les nœuds du cluster via `docker stack ps`

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main)
$ docker stack deploy -c docker-compose.yml travelo_stack
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Updating service travelo_stack_backend (id: s5b8y04nn9s54at3vscxzny4i)
Updating service travelo_stack_frontend (id: rznn3s1lu98lty49w6kk6pwgw)
Updating service travelo_stack_database (id: poaxoed26x7ipouva7w07kme4)
```

FIGURE 59 – Mise à jour déclarative de la stack vers 2 réplicas pour chaque service

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main)
$ docker stack services travelo_stack
ID             NAME                    MODE         REPLICAS   IMAGE                                PORTS
s5b8y04nn9s5   travelo_stack_backend   replicated   2/2        localhost:5000/travelo-backend:1.0   *:8080->8080/tcp
poaxoed26x7i   travelo_stack_database  replicated   2/2        mysql:8.0
rznn3s1lu98l   travelo_stack_frontend  replicated   2/2        localhost:5000/travelo-frontend:1.0  *:3000->3000/tcp
```

FIGURE 60 – État de la stack confirmant `replicas: 2/2` pour le backend, le frontend et la base de données

```yaml
deploy:
  mode: replicated
  replicas: 2
  resources:
    limits:
      cpus: '0.50'
      memory: 512M
```

FIGURE 61 – Extrait YAML de limitation stricte des ressources CPU et mémoire (Q15)

```yaml
deploy:
  mode: replicated
  replicas: 2
  resources:
    limits:
      cpus: '0.50'
      memory: 512M
  restart_policy:
    condition: on-failure
    delay: 5s
    max_attempts: 3
    window: 120s
  placement:
    constraints:
      - node.role == manager
```

FIGURE 62 – Extrait YAML de la politique de redémarrage résiliente (`restart_policy`) (Q16)

3.3 Q17. Configuration du Reverse Proxy (Nginx / Traefik)

Afin de centraliser les flux et masquer l'architecture interne des microservices, nous intégrons un Reverse Proxy agissant comme **Single Entry Point (API Gateway)**.

3.3.1 Fichier de configuration Nginx (nginx.conf)

```
events {}
http {
    upstream backend_servers {
        server backend:8080;
    }
    upstream frontend_servers {
        server frontend:3000;
    }
    server {
        listen 80;
        # Routage vers l'API Backend
        location /api/ {
            proxy_pass http://backend_servers/api/;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }
        # Routage vers le Frontend React
        location / {
            proxy_pass http://frontend_servers;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }
    }
}
```

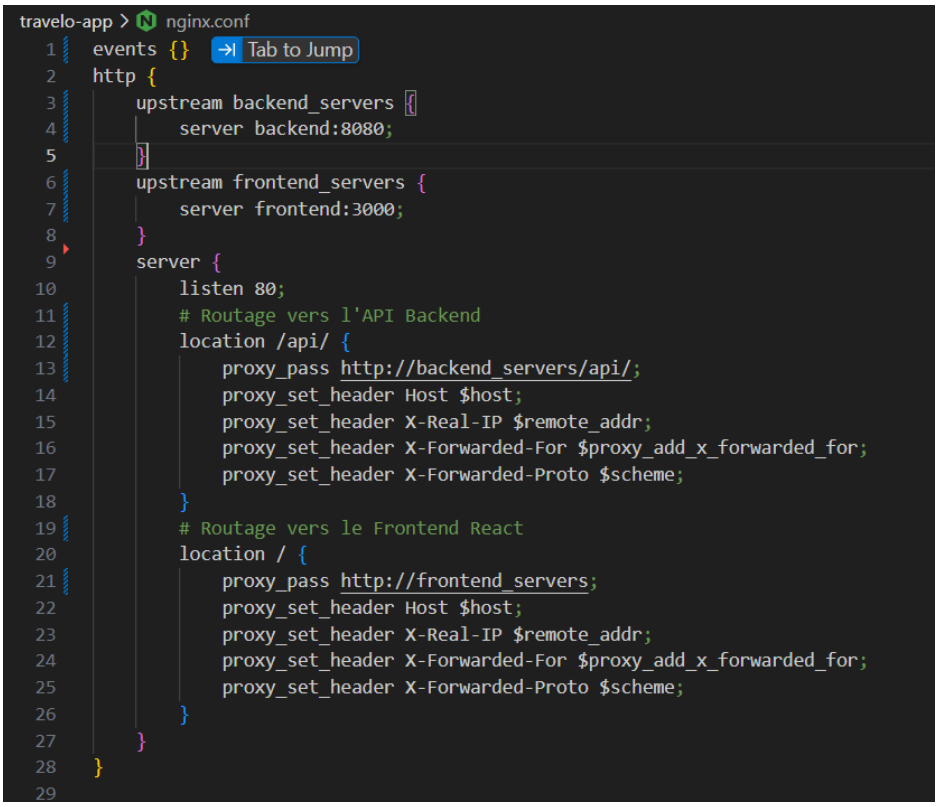


FIGURE 63 – Configuration des upstreams et des règles de routage proxy dans `nginx.conf`



FIGURE 64 – Déclaration du service de proxying rattaché au réseau overlay dans la stack Swarm

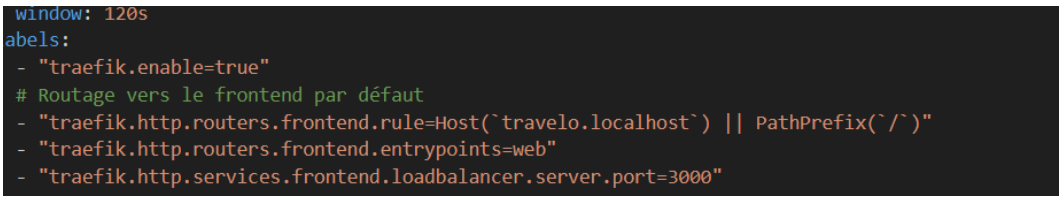


FIGURE 65 – Alternative étudiée : Routage par étiquettes (*labels*) dynamiques avec Traefik

3.4 Q18. Validation finale du Cluster Swarm dans le navigateur

L'infrastructure complète est validée de bout en bout en exploitant le point d'entrée unique sur le port standard HTTP (80).

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main)
$ docker stack services travelo_stack
ID             NAME                      MODE         REPLICAS   IMAGE                                  PORTS
s5b8y04nn9s5   travelo_stack_backend     replicated   2/2        localhost:5000/travelo-backend:1.0
poaxoed26x7i   travelo_stack_database    replicated   2/2        mysql:8.0                              *:3307->3306/
tcp
rznn3s1lu98l   travelo_stack_frontend    replicated   2/2        localhost:5000/travelo-frontend:1.0
7sy9xbv71dlp   travelo_stack_proxy       replicated   1/1        nginx:alpine                           *:80->80/tcp
```

FIGURE 66 – Inventaire `docker stack services` : les 4 services (dont le proxy sur le port 80) sont opérationnels

```
anase@Anas MINGW64 ~/OneDrive/Bureau/ASEDS INE2/Conteneurisation des APP/TP-1/travelo-app (main)
$ docker stack ps travelo_stack
0jlqhmbgan3t    \_ travelo_stack_database.2   mysql:8.0                          docker-desktop   Shutdown
  Failed 43 seconds ago     "task: non-zero exit (1)"
d2gh0lj9sojn    \_ travelo_stack_database.2   mysql:8.0                          docker-desktop   Shutdown
  Failed 2 minutes ago      "task: non-zero exit (1)"
8pmcu37nd1ds    \_ travelo_stack_database.2   mysql:8.0                          docker-desktop   Shutdown
  Failed 4 minutes ago      "task: non-zero exit (1)"
sdzr5934egzx    travelo_stack_frontend.1      localhost:5000/travelo-frontend:1.0   docker-desktop   Running
  Running 39 minutes ago
7hgkfv4uaha2    \_ travelo_stack_frontend.1   localhost:5000/travelo-frontend:1.0   docker-desktop   Shutdown
  Shutdown 40 minutes ago
i1ibuinv0tb3    travelo_stack_frontend.2      localhost:5000/travelo-frontend:1.0   docker-desktop   Running
  Running 40 minutes ago
maa1p6im288k    \_ travelo_stack_frontend.2   localhost:5000/travelo-frontend:1.0   docker-desktop   Shutdown
  Shutdown 40 minutes ago
zcwuh7noge1h    travelo_stack_proxy.1         nginx:alpine                       docker-desktop   Running
  Running 8 minutes ago
```

FIGURE 67 – Surveillance des tâches `docker stack ps` validant l'équilibrage et l'auto-guérison Swarm

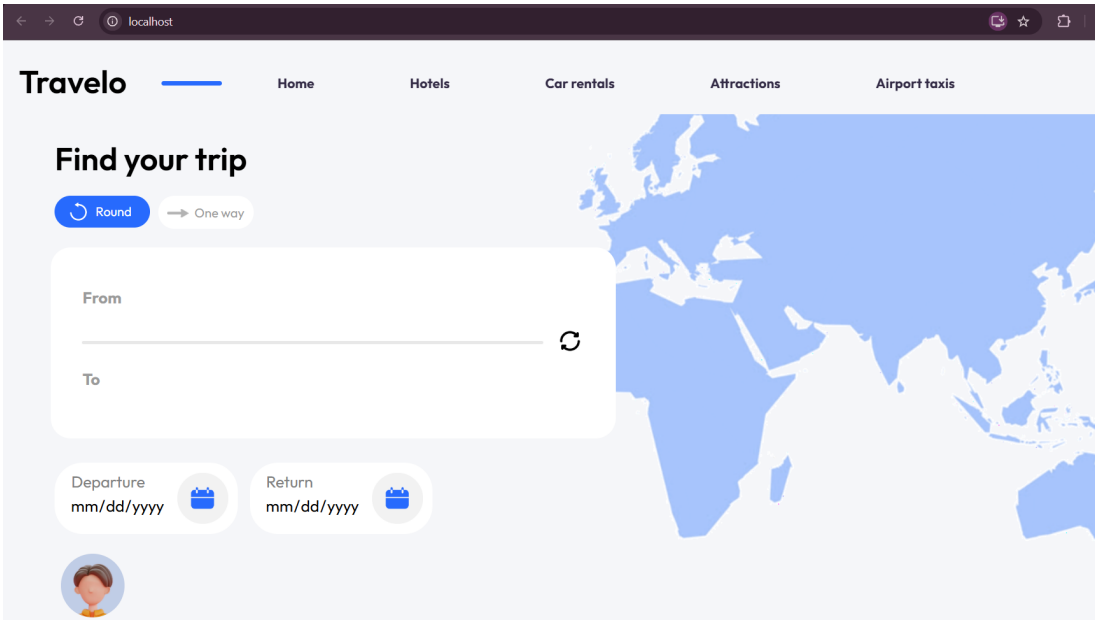


FIGURE 68 – Accès direct au Frontend via le Reverse Proxy sur `http://localhost` (port standard 80)

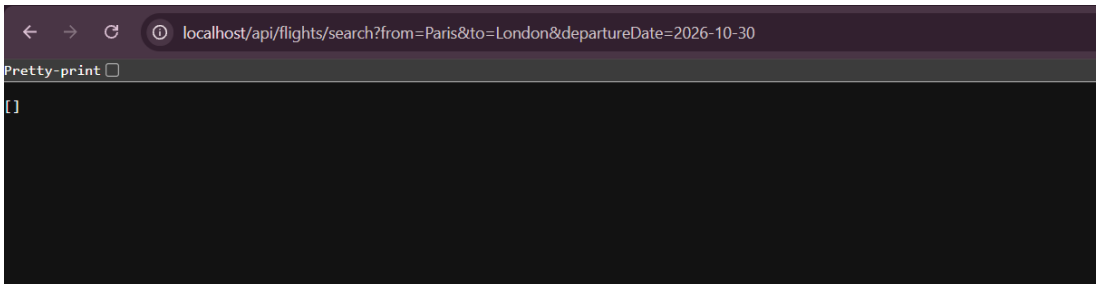


FIGURE 69 – Validation du routage de l'API Backend via `http://localhost/api/flights/search?...`

4 Conclusion Générale

Le Proof of Concept mené pour le compte de l'entreprise **NeoTech** démontre l'efficacité absolue de l'approche conteneurisée pour surmonter les problématiques d'hétérogénéité des environnements et de non-portabilité logicielle :

- **Portabilité et Parité Dev/Prod :** L'application 3-tiers Travelo est désormais encapsulée dans des artefacts immuables, garantissant un comportement strictement identique sur tout système hôte supportant Docker.
- **Excellence DevSecOps :** La mise en œuvre de *multi-stage builds* et l'adoption d'images allégées (`eclipse-temurin:23-jre-alpine` et `nginx:1-alpine-slim`) ont permis d'éradiquer la totalité des vulnérabilités critiques détectées par Docker Scout.
- **Souveraineté des artefacts :** Le déploiement du registre privé d'entreprise avec interface d'administration Web offre une autonomie complète vis-à-vis des dépôts distants.
- **Haute Disponibilité & Tolérance aux pannes :** L'orchestration avec Docker Swarm assure une répartition de charge transparente par adresse IP virtuelle (VIP), une limitation stricte des ressources pour éliminer l'effet *noisy neighbor*, et une auto-guérison instantanée orchestrée par Nginx en passerelle unifiée.